

DINEFLOW

Multi-Restaurant Ordering & Kitchen Operations Platform

Senior Developer Product Requirements & Technical Implementation Brief

Document detail	Value
Working product name	DineFlow (provisional; naming and domain availability not verified)
Document version	1.0 — developer handoff brief
Intended first market	University/campus communities in Kenya; initial pilot around Kabarak University is a proposed starting point
Primary delivery target	Responsive web application / installable PWA
Recommended baseline stack	Next.js, TypeScript, Tailwind CSS, Supabase/PostgreSQL, realtime updates, payment-provider integration
Document status	Product and engineering specification; open business decisions are listed near the end

Purpose

This brief describes what to build, who uses it, how the workflows behave, the information the system stores, and the minimum quality/security criteria for launch. It is intended to let a senior developer estimate, design, implement and test the first production-quality version without needing the original verbal explanation.

Important interpretation

DineFlow is not a collection of separate restaurant websites. It is one multi-tenant platform with a shared codebase and database, where each restaurant has a configurable public storefront and a restricted operations workspace. The product name is provisional; architecture and requirements must not depend on that name.

Contents

- 1. Executive summary and product vision
- 2. Goals, non-goals and product principles
- 3. User roles and access boundaries
- 4. End-to-end user journeys
- 5. Functional requirements: student marketplace
- 6. Functional requirements: restaurant workspace
- 7. Functional requirements: platform administration
- 8. Orders, preparation estimates and status model
- 9. Payments, refunds and settlement
- 10. Data model and relationships
- 11. Technical architecture and implementation conventions
- 12. Security, privacy and compliance considerations
- 13. Notifications and user-facing copy
- 14. Analytics, reporting and audit trail
- 15. Edge cases and failure handling
- 16. Non-functional requirements
- 17. Acceptance criteria and test plan
- 18. Phased delivery plan
- 19. Decisions to confirm before production
- Appendix A. Suggested screen and route inventory
- Appendix B. Definition of done

How to read this brief: requirements described as **MUST** are launch-critical. **SHOULD** items are strongly recommended but may be scheduled after the first controlled pilot. **LATER** items are outside the minimum viable product unless business validation makes them urgent.

1. Executive summary and product vision

DineFlow is a campus-oriented, multi-restaurant discovery and advance-ordering platform. Students use one marketplace to discover participating restaurants, browse their current menus, select items, place an order, pay through an approved payment flow, and receive an estimated ready time. They travel to the restaurant when the food is close to ready, collect it using a short order code or other verification mechanism, and avoid unnecessary waiting in queues.

Restaurants receive a dedicated workspace to accept and prepare incoming orders, keep menus and availability updated, provide realistic preparation estimates, communicate delays, and review sales/order summaries. The platform operator receives a central administrative console to onboard many independent restaurants through reusable templates, apply branding and operational configuration, manage users, publish storefronts, and monitor platform-wide activity.

The primary differentiator is the combination of multi-restaurant campus discovery, advance pickup/dine-in ordering, preparation-time visibility, and a repeatable onboarding workflow for independent businesses. The MVP should prioritize dependable order handling and kitchen adoption over advanced recommendations or delivery logistics.

1.1 Product outcomes

- Students can quickly answer: what food is available, where can I get it, how much does it cost, and when can I collect it?
- Restaurants can receive orders in a structured digital queue and reduce ad hoc order-taking and avoidable waiting.
- The platform operator can add a new restaurant without creating a separate codebase, deployment or manually built website.
- Payment and order statuses remain accurate, auditable and resilient to late callbacks, duplicate requests, network loss and cancellations.
- The system can grow from a small campus pilot to multiple locations while respecting tenant data boundaries.

1.2 Main product surfaces

Surface	Audience	Primary purpose
Student marketplace	Students and other customers	Discover restaurants, browse menus, place and track orders.
Restaurant workspace	Owner, manager, kitchen/counter staff	Receive and process orders; manage menu, availability and basic reports.
Platform admin console	Platform owner and authorized platform staff	Onboard and configure businesses, manage access, review orders/payments and operate the marketplace.
Backend and integrations	No direct end-user interface	Authentication, tenant isolation, order state, payment verification, notifications, analytics and audit logs.

2. Goals, non-goals and product principles

2.1 MVP goals

- Provide one searchable marketplace containing multiple independent restaurant profiles.
- Generate restaurant pages from stored configuration rather than bespoke page code.

- Support menu categories, menu items, prices, photos, availability and simple item customizations.
- Support a student cart, checkout, order confirmation, status tracking and order history.
- Support restaurant acceptance/rejection, preparation-time estimates, kitchen status updates and pickup verification.
- Support an auditable payment state machine and at least one agreed payment mode suitable for the pilot.
- Provide secure, role-based access for platform administrators, restaurant owners/managers and restaurant staff.
- Provide basic platform and restaurant-level reporting and a usable mobile-first interface.

2.2 Explicit non-goals for the first release

- No in-house delivery fleet, driver dispatch or live driver tracking.
- No independent native Android or iOS apps at launch; use a responsive web app/PWA.
- No AI-based demand forecasting or machine-learned preparation estimates.
- No full stock/inventory/recipe costing system, supplier purchasing or accounting package.
- No complex loyalty points, coupons, memberships, table reservations or multi-restaurant single-cart checkout.
- No automatic creation of arbitrary custom software per business. The system uses a controlled set of configurable themes, fields and templates.

2.3 Product principles

- Mobile first: the customer journey must be comfortable on a phone and workable on low-to-moderate bandwidth.
- One cart, one restaurant: an order belongs to exactly one restaurant in the MVP.
- Trust payment events: the server verifies provider callbacks; the browser never determines that payment succeeded.
- Kitchen control: estimates are suggested by the system and confirmed or adjusted by restaurant staff.
- Tenant isolation: restaurant users must never access another restaurant's private orders, customers, settings or reports.
- Configurable, not hard-coded: branding, contact information, opening hours and menus are database-driven.
- Operational simplicity: large touch targets, clear order statuses, visible sound/visual alerts and few clicks for kitchen staff.

3. User roles and access boundaries

Role	Scope	Permitted actions	Restrictions
Anonymous visitor	Public marketplace	Browse published restaurants, menus, prices, hours and availability.	Cannot place an order until the chosen authentication rule is satisfied; cannot access private order details.
Student/customer	Own account	Manage own profile; create and view own orders; submit payment; track own orders; request cancellation where permitted.	Cannot edit restaurant menus, change prices, see other customers' records or access restaurant/admin workspaces.
Restaurant owner	One or more explicitly assigned restaurants	Manage assigned restaurant settings, menu, opening hours, orders, staff and reports; invite staff if enabled.	Cannot view another restaurant or platform-wide financial/administrative data.
Restaurant manager	Assigned restaurant(s)	Manage service availability, orders, menu and operational settings according to granted permissions.	Cannot transfer ownership, change platform-level fees or access unassigned tenants.

Role	Scope	Permitted actions	Restrictions
Kitchen/counter staff	Assigned restaurant	View relevant incoming orders, accept/advance/mark ready/collected, adjust prep estimates if permission allows.	Cannot see platform-level analytics, manage billing or change sensitive business settings by default.
Platform support/operations	Platform-wide, permission-limited	Review onboarding, moderate listings, inspect operational status and assist with disputes according to permission.	Actions with financial or account impact must be permissioned and audited.
Platform super-admin	Entire platform	Manage global settings, business onboarding, access, platform reporting and integration configuration.	High-privilege actions must be logged; secrets must never be shown in plaintext.

Authorization requirement: the backend must enforce permissions on every sensitive query and mutation. Hiding a button in the frontend is not a security control. A user's restaurant membership and role must be resolved from trusted server-side data.

3.1 Account and onboarding assumptions

- Student authentication: choose one launch approach—email/password, email magic link, phone-based login, or guest checkout with a verified contact. Do not require a university email until that rule is explicitly agreed.
- Restaurant owners are invited by a platform administrator or claim an already-created business through a verified onboarding process.
- Each staff membership is tied to a user ID, restaurant ID and role. Revoke access immediately when membership is removed.
- Platform super-admin privileges must not be self-granted through public registration.

4. End-to-end user journeys

4.1 Student ordering journey

1. Student opens the marketplace and sees participating restaurants that are published, currently accepting orders and within the selected service area (if location filtering is enabled).
2. Student searches or filters by restaurant, food, category, price, availability and optionally distance or estimated collection time.
3. Student opens a restaurant profile, reviews its location, current status, opening hours, menu categories, item prices, descriptions, photos and any preparation notes.
4. Student chooses an item, sets quantity and required options, adds optional notes, and adds it to the cart.
5. Cart shows item subtotals, item options, any service/platform fee, applicable tax only if configured, and the total payable. One restaurant per cart.
6. Student chooses order mode: pickup or dine-in. For dine-in, the selected restaurant must define how the customer will identify themselves or where/how seating is handled. The MVP should not imply a table reservation unless that feature is implemented.
7. Student confirms contact details, reviews policies and initiates payment or selects another allowed payment method.
8. Backend creates a pending order and a unique human-readable order number. Order is not treated as paid until payment is verified, unless the selected payment mode does not require prepayment.
9. Once payment is valid, restaurant receives the paid order. Restaurant accepts/rejects and confirms a ready-time estimate.

10. Student sees current status, estimate, restaurant directions, order code and the next meaningful action. When the restaurant marks the order ready, the student is notified.
11. Student presents the pickup code/order number, staff verifies collection, and the order becomes completed. Student can view it in order history.

4.2 Restaurant setup and daily operations

1. Platform admin creates a draft restaurant profile using the setup wizard and assigns the initial owner/manager.
2. Admin enters business identity, logo, colours, location, opening hours, supported order modes, menu categories, items, item options and preparation defaults.
3. Admin previews the generated public storefront as a mobile and desktop layout, fixes issues, and publishes it.
4. Owner signs in, confirms the profile and learns the order workflow. Restaurant can pause new orders, close temporarily, mark items unavailable and update menu prices.
5. Kitchen/counter dashboard shows incoming orders with prominent sound/visual cues; staff opens an order, reviews notes/options and accepts or rejects it with a reason.
6. On acceptance, staff confirms or changes the estimated ready time. The system informs the customer and records who changed the estimate and when.
7. Staff marks order preparing, then ready, then collected. A pickup code or equivalent check reduces incorrect handovers.
8. Owner reviews order volume, gross sales, payment status, cancelled/rejected orders and preparation-time accuracy.

4.3 Platform administrator onboarding journey

1. Open Admin > Restaurants > Add Restaurant.
2. Enter required identity and location fields; system checks duplicate slug/name or provides a warning where similar records exist.
3. Choose a theme/template; set logo, colours and optional header/cover image.
4. Configure hours, temporary closure, pickup/dine-in support, default prep behaviour, accepted payment methods and customer instructions.
5. Build menu categories and items. Each item has price, active/available status, optional image, description, preparation time and optional modifiers.
6. Assign owner/manager accounts and permissions. The invite must be verifiable and expire or be revocable.
7. Preview, save as draft, run required-field validation, publish when ready.
8. Admin can pause/suspend a restaurant, unpublish it, or reactivate it with an audit reason. Pausing new orders must not silently erase existing orders.

5. Functional requirements: student marketplace

5.1 Home / restaurant discovery

The home screen is a marketplace, not a list of static links. It must be optimized for mobile screens and must only show public information for published businesses.

Element	Requirement / expected behaviour
Top bar	Product mark, location/service-area selector if supported, search input, account/orders entry point and cart indicator.
Intro text	Suggested copy: “What are you craving?” Supporting copy: “Order ahead, then collect when it’s ready.” The copy may be localized later.

Element	Requirement / expected behaviour
Search	Search restaurant names, menu item names and categories. Handle empty searches and no-result states.
Restaurant cards	Logo/cover image, name, short description or cuisine tags, open/closed/paused status, approximate distance if supported, price cue if configured, and order-time indicator when reliable.
Filters	Food/category, maximum price, currently accepting orders, vegetarian or dietary labels only when maintained by the business, and optional distance/time filters.
Sort	Recommended/default, nearest when location is available, price, or estimated readiness where reliable.
Empty/error states	If no restaurants are available, explain why and offer to clear filters or view all restaurants. Do not fabricate availability.
Business status	Closed or paused businesses remain viewable if desired, but ordering must be disabled and the status explained. Platform admin controls the visibility rule.

5.2 Restaurant profile and menu

- Header: restaurant logo, name, optional cover image, description, category/cuisine tags, map/location/directions, current business status and opening hours.
- Ordering details: order modes accepted, pickup instructions, minimum lead time if configured, and any disclosed service or payment charges.
- Menu sections: category tabs or scrollable groups, item image, name, description, price, availability and preparation estimate where appropriate.
- Menu search and category navigation should work on small screens without hiding cart actions.
- Unavailable items must be clearly labelled and cannot be added to cart. A menu item that becomes unavailable while in cart must be revalidated at checkout.
- Item details may offer a single or multiple choice modifier group, e.g. size, side choice or add-ons. Modifier requirements, min/max selections and additional costs are configured by the restaurant.
- Item notes may be allowed as free text, but the interface must say that the kitchen may not be able to honor every request.
- Restaurant-specific content must be loaded by restaurant ID/slug and checked server-side. Never trust a client-supplied restaurant ID for authorization.

5.3 Cart

- The cart contains line items, chosen modifiers, quantity, item price snapshots, notes and calculated line totals.
- One active cart may contain items from one restaurant only. If a customer chooses another restaurant, prompt: “Your cart contains items from [Restaurant]. Start a new cart for [Other Restaurant]?” Preserve or clear only after a deliberate choice.
- Allow item quantity changes and removal. Recalculate totals on the server at checkout, using current prices and configuration.
- Show subtotal, fees, discount/tax only if enabled, and total before payment. Label every fee transparently.
- Persist cart across page refreshes for signed-in users if practical. Never treat a locally stored total as authoritative.
- If a menu item price or availability changes, explain the change and require the customer to review before placing the order.

5.4 Checkout

Checkout must be short and explicit. Required fields should be limited to information needed to fulfill and reconcile the order.

Field / control	Rules
Contact / identity	Use authenticated profile or collect required name and a verified/contactable phone or email according to the launch policy.
Order mode	Pickup or dine-in, limited to modes enabled by the selected restaurant.
Requested timing	MVP default: order as soon as possible. A scheduled order feature is out of scope unless explicitly added.
Order note	Optional customer instruction; length-limited and sanitized. Do not use it for payment details or sensitive information.
Payment method	Show only methods available for that restaurant and the platform's approved payment configuration.
Price summary	Final server-calculated item subtotal, add-ons, fees and total in KES (unless multi-currency is intentionally supported later).
Policies	Show cancellation/refund guidance and a clear place-order/pay action before commitment.
Submit	Protect against double-click/refresh duplicates with an idempotency strategy; show a progress state and recoverable errors.

5.5 Order tracking and history

- Confirmation screen shows order number, restaurant name and location, order contents, paid/unpaid status, current status, ETA/ready time if available, order mode and pickup code.
- Live tracking displays a human-readable timeline: Order received / Awaiting restaurant confirmation / Accepted / Preparing / Ready for collection / Collected. Show cancellation or refund status when applicable.
- ETA must display the time zone/local time and a “last updated” time when changed. If an estimate is not yet confirmed, state “Waiting for restaurant to confirm preparation time.”
- Customer can call or message the restaurant only through contact methods the business chooses to publish; the app should not expose private owner details.
- Order history lists prior orders with restaurant, date, total, status and “View details.” Reorder may be offered only after revalidating current price and availability.
- Customer can request cancellation only while the order is in an eligible state and under the configured policy. UI must explain when cancellation requires restaurant approval.

5.6 Profile and accessibility

- Profile: name and contact details, authentication/security controls, order history, notification preferences, sign out, and privacy/help links.
- Touch targets must be comfortable on mobile; text and status must not rely on colour alone.
- Provide keyboard access and visible focus on web, labels for form inputs, helpful error messages and sufficient text contrast.
- Use plain language, avoid unnecessary screens, and make the primary action clear on every ordering screen.

6. Functional requirements: restaurant workspace

6.1 Operational dashboard

The restaurant dashboard is the working tool for a busy owner, counter attendant or kitchen operator. The live order queue should be the primary workspace, not a secondary analytics page.

- Show today's order count, orders requiring action, orders in preparation, ready orders awaiting collection and payment exceptions.
- Provide a prominent "Pause new orders" / "Resume orders" control with confirmation and reason (optional).
- Show open/closed state derived from operating hours plus manual pause/closure overrides.
- Provide visual and optional audible alerts for new orders; allow staff to acknowledge/read an order to stop repeated alerts without hiding it from the queue.
- Show network/realtime connection state. If live updates disconnect, refresh/retry and do not pretend the queue is current.
- Support a simple tablet/desktop layout with large action buttons; orders must remain usable on mobile.

6.2 Incoming order queue and order detail

Feature	Requirement
Queue groups	New/pending action, accepted/preparing, ready for collection, completed/rejected/cancelled. Archive older orders but keep searchable history.
Order card	Order number, received time, items summary, mode, payment state, requested notes, current state and action deadline/age.
Order detail	Full item names/options, quantities, allergen/dietary information only if configured, customer instructions, total, payment reference/status, timestamps and audit timeline.
Accept	Restaurant confirms it can fulfill the order and must provide a ready-time estimate or choose a configured default subject to confirmation.
Reject	Require a reason such as item unavailable, kitchen too busy, restaurant closing, or other. Backend sets the correct order/payment path and informs customer.
Prep estimate	Allow quick preset buttons such as 5, 10, 15, 20, 30 minutes plus custom value within an admin-configured range. Exact presets are configurable.
Status changes	Only valid transitions are permitted. Store actor, timestamp and optional reason for each transition.
Ready/collection	Mark "Ready for collection", then "Collected" after verifying order code or another agreed method. Avoid accidental completion with a confirmation step or undo policy.
Delay	Let staff update the estimated ready time, select/explain a reason and notify the customer. Record previous and new ETA for performance measurement.
Concurrency	If two employees update an order at once, avoid inconsistent status transitions; return the latest state and explain conflicts.

6.3 Menu management

- Create, edit, reorder and archive menu categories.

- Create, edit and archive menu items with name, description, image, category, price in KES, active flag, availability flag, preparation-time default and optional tags.
- Configure item modifiers/options with choices, extra prices, min/max selections and required/optional rules.
- Quickly mark an item “Unavailable” and restore it without deleting its definition or historical order records.
- Price edits apply to future orders; historical order line items preserve their purchased names, options and price snapshots.
- Validate that prices are non-negative and required fields exist before publishing an item.
- Image upload must enforce file type, file-size constraints and safe storage paths. Optimize images for mobile.

6.4 Restaurant settings and team management

- Edit allowed business details: description, logo/cover, contact channels, address/map pin, menu, hours and pickup instructions. Some sensitive fields may require platform review.
- Manage recurring opening hours per day and exceptional dates/temporary closures. UI must handle overnight hours only if implemented intentionally.
- Configure supported order modes, prep-time defaults/ranges, new-order alert preference, and customer-facing notes.
- Owner can invite staff, assign limited roles and remove memberships. Invitation is tied to the intended email/phone and expires or can be revoked.
- Payment onboarding/configuration should be handled through the approved provider process; raw provider secrets must never be entered into ordinary form fields or displayed back.

6.5 Reporting

- Order count and gross order value for a selected date range.
- Counts of completed, rejected, cancelled and refunded orders.
- Item-level sales quantity and sales value, based on order line snapshots.
- Average/median preparation duration where timestamps support it, percentage of orders ready by the last communicated ETA, and count of ETA changes.
- Payment reconciliation view with order number, expected total, provider reference, payment status and any exception requiring attention.
- Reports must be scoped to the selected restaurant and respect staff permissions. Clearly label gross sales versus net settlements/fees; do not present these as accounting-grade profit data.

7. Functional requirements: platform administration

7.1 Admin overview

The platform admin console manages the marketplace as a whole. It must make onboarding efficient and operational problems visible without granting every employee unrestricted access.

- Global summary: total restaurants, published/paused restaurants, orders today, order/payment exceptions and basic transaction totals.
- Quick actions: Add Restaurant, View Restaurants, Review Orders, View Payment Exceptions and Manage Users (permission dependent).
- Platform alerts: provider webhook failures, pending payment exceptions, restaurant onboarding incomplete, unusually high rejection/cancellation rates.

7.2 Restaurant list and onboarding wizard

Each business record should move through a controlled lifecycle: Draft → Ready for review (optional) → Published/Active → Paused or Suspended → Archived. Archived records are retained for audit/history and are not deleted merely because a business stops participating.

Wizard step	Required fields / behaviour
1. Identity	Legal/trading name if collected, public display name, slug, business description, cuisine/category labels, public contact details.
2. Branding	Logo, optional cover image, primary/secondary/accent colours, text/contrast preview and selected layout template.
3. Location	Address, locality/campus/service area, map pin or location string, directions text. Distance sorting requires a reliable coordinate or other defined location source.
4. Hours & operations	Hours for each day, exception/closure controls, pickup/dine-in mode, pickup instructions, acceptance/pause behaviour.
5. Menu builder	Categories, items, prices, images, descriptions, availability, preparation defaults, modifier groups and validation.
6. Payment setup	Approved payment method(s), provider/merchant mapping and onboarding state. Secrets stay in backend-managed secret storage.
7. Owner & staff	Invite owner and optional manager/staff; enforce unique membership and proper permissions.
8. Preview & publish	Mobile/desktop preview, validation checklist, save draft, publish, suspend/unpublish and change log.

7.3 Storefront template engine

- Public storefront is generated at runtime from a restaurant configuration record and its active menu data.
- Use a fixed, tested design system with a small set of responsive layout variants. Colour and imagery are configurable within safe constraints; layout is not arbitrary per restaurant in the MVP.
- Prevent unreadable branding: check text/background contrast or warn admin, retain system defaults for missing values and do not allow custom CSS or raw HTML from restaurant records.
- Slug is unique and URL-safe. Changing a published slug should either require confirmation and support a redirect, or be disallowed in the MVP to avoid broken links.
- Preview mode must not expose draft restaurants to the public. Public APIs must return only explicitly published and visible fields.

7.4 Platform-wide operations

- Search/filter restaurants by name, status, location and onboarding completeness.
- Inspect order/payment metadata for support purposes using permissioned access. Sensitive data is minimized and access is logged.
- Suspend/restore a restaurant with a reason; suspension blocks new ordering but preserves orders, financial records and required customer service handling.
- Manage global configuration: currencies, fee/tax policy, payment modes, order cancellation defaults, notification templates and supported service areas.
- Manage administrator/staff role assignments and review audit logs.
- Provide a platform-level reconciliation view for all payment events and unresolved exceptions.

8. Orders, preparation estimates and status model

8.1 Order lifecycle

Use a server-controlled state machine. Exact naming may vary internally, but the user-facing status must be clear and every transition must be validated.

State	Meaning	Allowed next states / rules
pending_payment	Order draft/commitment created and payment not yet confirmed.	paid/awaiting_restaurant after verified success; payment_failed/expired/cancelled based on rules. No kitchen fulfilment should begin for prepaid orders before payment is valid unless explicitly authorized.
awaiting_restaurant	Payment is confirmed or a valid non-prepaid mode was selected; restaurant response is pending.	accepted, rejected, or cancelled under policy. Apply a response timeout if configured and alert staff.
accepted	Restaurant confirmed the order and supplied the current ETA.	preparing, ready, or cancelled only if the policy permits and the action is audited.
preparing	Kitchen is preparing the order.	ready_for_collection; cancellation normally requires restaurant/admin handling.
ready_for_collection	Restaurant reports the order is ready.	collected/completed; may return to preparing only through a correction flow with audit if staff made a mistake.
collected/completed	Customer collected the order; fulfilment is complete.	Terminal for normal workflow. Refund/dispute is a separate financial process, not a status rewrite.
rejected	Restaurant cannot fulfill the order.	Terminal for fulfilment; trigger payment reversal/refund path where money was collected.
cancelled	Order was cancelled under an allowed rule.	Terminal for fulfilment; financial status tracked separately.
expired/failed	Checkout/payment/acceptance expired or failed under configured rules.	Terminal for normal fulfilment unless an explicit recovery/retry flow is defined.

Recommended separation: order_status and payment_status are different fields/state machines. An order can be cancelled while its refund remains pending; the user must see both facts rather than a misleading single status.

8.2 Preparation-time estimation

MVP rule: each menu item has an optional baseline preparation duration. On order acceptance, staff is shown a suggested duration derived from the longest item path or another documented combination rule plus a simple queue/load adjustment. Staff confirms or edits the estimate. Store estimates as timestamps in UTC and display them in the restaurant/customer local timezone.

A practical first implementation may use: suggested prep minutes = maximum baseline preparation time among order items + a configurable workload buffer based on active orders. If the kitchen chooses a more sophisticated sum/parallel-station model, document it and keep the staff override. Do not advertise the number as AI-powered.

- ETA is first considered confirmed when the restaurant accepts the order or explicitly sets it; before that, show a waiting message.
- If ETA changes, store the previous value, new value, actor, timestamp and optional reason.
- When ready, status overrides the estimated time; do not keep showing a future ETA after the meal is ready.

- Capture `accepted_at`, `preparing_at`, `ready_at` and `collected_at` to calculate actual durations.
- If a kitchen misses an ETA, prompt staff to update it when practical and notify the student; avoid spamming repeated notifications.

8.3 Pickup verification

- Generate a random, non-sequential pickup code/order token at order creation. Do not use a database primary key as a secret.
- Show the code to the customer only after the appropriate order/payment state. Show staff enough order context to avoid handing a meal to the wrong person.
- Staff may enter or verify the code, or use a controlled “mark collected” action. The exact MVP method should be selected during design.
- Rate-limit code attempts and do not expose other customers’ details after an invalid entry.

9. Payments, refunds and settlement

9.1 Payment rules

- All money calculations and order totals are computed server-side using integer minor units (or another explicitly safe decimal strategy); never rely on browser-calculated totals.
- Store currency code with every financial transaction. MVP default: KES.
- Create a payment attempt with a unique internal ID and provider reference before redirecting/initiating provider interaction.
- Payment success is recorded only after server-side verification of the provider callback/status, matching order/amount/currency and handling replay/duplicate events idempotently.
- Never store card credentials or M-Pesa PINs. Do not log authentication secrets, provider tokens or full sensitive payloads.
- If payment remains pending, display “Confirming payment” and poll/refresh or await callback. Do not create a second charge because the browser timed out.
- Capture provider references and a reconciliation history sufficient for support to match the payment to the order.
- Orders and payment records must remain auditable even when a restaurant is suspended or a customer account is deleted/anonymized.

9.2 Recommended initial payment decision

Before live launch, the business owner must decide whether the platform will: (A) enable provider-supported direct payments/settlement to each restaurant, (B) use a supported marketplace/split-settlement arrangement, or (C) begin the pilot with pay-at-pickup/manual payment while online payment is validated. Provider capabilities, merchant onboarding, fees, settlement responsibility, chargebacks, refunds, tax/accounting treatment and applicable Kenyan legal requirements must be confirmed with the provider and qualified local advisers.

Do not hard-code a money flow

Build a payment-provider abstraction so the order system does not assume that all funds belong to the platform operator. The MVP can launch with one approved method, but the data model should record the beneficiary/merchant mapping and financial events clearly.

9.3 Refund, cancellation and reconciliation

- Define cancellation eligibility by order state and disclose the policy before payment. Default recommendation: customer self-cancellation is available only before restaurant acceptance; later cancellation goes through restaurant/admin review.

- If a prepaid order is rejected or cancelled, create a refund/reversal record and show “Refund initiated” / “Refund completed” based on provider-confirmed state, not merely a button click.
- Refunds must be idempotent and linked to the original payment. Partial refunds should be out of scope unless required by launch operations.
- Provide a payment exception queue: pending too long, amount mismatch, duplicate callback, paid order not delivered to restaurant, rejected paid order with refund pending.
- Use daily reconciliation exports or reports that compare internal payment records with provider references/statuses. Track who resolves manual exceptions.

10. Data model and relationships

The following is a logical schema proposal, not a mandate for exact table names. The developer should normalize relational data, add indexes based on real access patterns and provide database migrations from the beginning. Use UUIDs for internal IDs unless there is a documented reason not to. Keep public slugs and customer-facing order numbers separate from database IDs.

Entity / table	Important fields	Notes / relationships
users / profiles	id, display_name, email/phone, authentication reference, status, created_at	User identity/profile. Avoid duplicating provider-managed password data.
restaurants	id, slug, display_name, legal/trading_name if needed, description, logo_url, cover_url, theme tokens, address, latitude/longitude, timezone, status, visibility, created_at, updated_at	One record per business tenant. Public profile fields must be explicitly selected for public responses.
restaurant_memberships	id, restaurant_id, user_id, role, status, invited_by, created_at	Links users to assigned tenants. Unique active membership per user/restaurant as appropriate.
restaurant_hours	id, restaurant_id, weekday, opens_at, closes_at, is_closed	Recurring weekly schedule; support exceptions/closures in a separate table or clear structured model.
restaurant_closures	id, restaurant_id, start_at, end_at, reason, created_by	Temporary closure or exceptional hours.
menu_categories	id, restaurant_id, name, description, sort_order, active	Categories owned by one tenant.
menu_items	id, restaurant_id, category_id, name, description, image_url, price_minor, currency, prep_minutes, active, availability_status, tags, sort_order	Price and item definition. Order records store immutable snapshots.
modifier_groups	id, restaurant_id, name, required, min_select, max_select, sort_order	A group of customization choices associated with menu items.
modifier_options	id, modifier_group_id, name, price_delta_minor, active, sort_order	Each choice can add cost; validate selection rules server-side.
menu_item_modifier_groups	menu_item_id, modifier_group_id, sort_order	Join table for reusable modifier groups if reuse is supported.
carts / cart_items	user/session reference, restaurant_id; item_id, quantity, selected modifier snapshot, note	May be transient or persisted. Not the financial source of truth.
orders	id, public_order_number, customer_id, restaurant_id, order_mode, order_status, payment_status, currency, item_subtotal_minor, fee_minor, tax_minor, total_minor, customer_note, pickup_code_hash/token, accepted_at, eta_at, ready_at, collected_at, timestamps	One order belongs to one restaurant. Financial totals and key names/options must be snapshotted.

Entity / table	Important fields	Notes / relationships
order_items	id, order_id, menu_item_id nullable, item_name_snapshot, unit_price_minor, quantity, modifiers_snapshot, note, line_total_minor	Historical snapshot survives later menu edits/deletions.
order_status_events	id, order_id, previous_status, new_status, actor_user_id/system, reason, occurred_at, metadata	Append-only audit timeline; control access to metadata.
payment_attempts / transactions	id, order_id, provider, provider_reference, merchant/beneficiary mapping, amount_minor, currency, status, initiated_at, confirmed_at	Do not overload order state to represent financial status.
payment_events	id, payment_id, provider_event_id, event_type, payload_hash or minimized payload, received_at, processed_at, processing_status	Unique provider event ID/idempotency key to suppress duplicate callbacks. Minimize stored sensitive payload.
refunds	id, payment_id, order_id, amount_minor, reason, provider_reference, status, requested_by, timestamps	MVP can restrict to full refunds; state must be provider-confirmed.
notifications	id, user_id, restaurant_id nullable, order_id nullable, channel, template_key, status, provider_ref, created_at, sent_at, error_code	Track delivery attempts without exposing sensitive content in logs.
platform_settings	key, value, type, updated_by, updated_at	Sensitive credentials must be in managed secrets, not ordinary settings rows.
audit_logs	id, actor_id, scope, scope_id, action, target_type, target_id, timestamp, reason, safe_metadata	Record high-impact actions; do not log secrets or unnecessary personal information.

10.1 Relationship and integrity rules

- Restaurant-owned rows include restaurant_id directly or are reachable through a parent with enforced tenant checks.
- An order has exactly one customer/account or supported guest identity, exactly one restaurant, one currency and one immutable checkout total.
- Order items snapshot display name, selected modifiers and prices at purchase time.
- A restaurant menu item ID may become null on historical order items if the item is removed; historical snapshot data must remain.
- Foreign keys and constraints should prevent orphaned orders, invalid quantities, negative prices and duplicate active membership records.
- Index likely query columns: restaurant_id + status + created_at; customer_id + created_at; slug; provider_reference; public_order_number; payment status.

11. Technical architecture and implementation conventions

11.1 Recommended baseline stack

Layer	Recommendation	Implementation notes
Web application	Next.js + TypeScript	Use a current stable version at project start; responsive UI for marketplace, restaurant workspace and admin console.
Styling/UI	Tailwind CSS + accessible component primitives	Reusable components, design tokens, responsive layouts and clear status styles.

Layer	Recommendation	Implementation notes
Database	Supabase PostgreSQL	Relational constraints, migrations, indexes, staging and production projects.
Auth	Supabase Auth or another explicitly selected provider	Server-verified identity; memberships and roles stored in trusted application tables.
Storage	Supabase Storage or equivalent object storage	Separate public storefront assets from private assets; validated uploads and derived image sizes.
Realtime	Supabase Realtime or server-sent/websocket layer	Use for order updates, with a fetch/refresh fallback and connection status.
Payments	Provider adapter around approved API integration	Provider-agnostic order interface; start with one approved method and merchant/settlement model.
Notifications	In-app first; email/SMS/WhatsApp/push only where configured and consented	Delivery failure should not roll back a committed order or payment.
Hosting	Vercel or equivalent managed hosting	Separate preview/staging and production; configure environment secrets and webhook URLs.
Monitoring	Structured server logs + error monitoring + uptime checks	Redact PII/secrets; alert on payment webhook failures and elevated order errors.

11.2 Application organization

Use clear domain modules, even if the initial codebase is a modular monolith. Avoid microservices for the MVP. Example domain boundaries: authentication/access, restaurants, menus, carts/checkout, orders/state machine, payments, notifications, reporting and platform administration.

- Use TypeScript types for domain objects and validated schemas for incoming API/form data.
- Keep authoritative calculations and state transitions in server-side functions/routes/actions, not only frontend components.
- Centralize restaurant access checks and order/payment state transition rules to avoid inconsistent implementations across screens.
- Use database migrations in source control; do not rely on manual production schema changes.
- Use environment variables/managed secrets for provider credentials and signing keys; never commit secrets.
- Use idempotency keys or equivalent duplicate protection for create-order/payment/refund actions.
- Add API rate limits or abuse protection to authentication, checkout, pickup-code validation and public search where appropriate.

11.3 API / command surface (illustrative)

Exact route style is an engineering choice, but the following capabilities must exist with authentication and authorization appropriate to each action:

Domain	Required operations
Public discovery	List/search published restaurants; fetch public restaurant profile; fetch categories/items/availability.
Customer	Read/update own profile; manage cart; create checkout/order; read own order; cancel/request cancellation; read own order history.

Domain	Required operations
Restaurant staff	List orders for assigned restaurant; read order details; accept/reject; set ETA; transition preparing/ready/collected; update menu and settings by role.
Platform admin	Create/update/publish/pause restaurant; assign staff; preview draft; query platform orders/payments; resolve payment exceptions; inspect audit events.
Payments/webhooks	Initiate payment; handle verified provider callback; query provider state when needed; create refund request; process provider event idempotently.
Notifications	Create and dispatch templates for order received/accepted/ETA changed/ready/cancelled/refund events; track delivery outcome.

11.4 Realtime behaviour

Realtime channels must be scoped so customers receive updates only for their own orders and restaurant users receive updates only for their assigned restaurant(s). Never broadcast private order/customer payloads on a channel that all signed-in users can subscribe to. Every client should be able to recover by fetching the current server state after reconnecting.

12. Security, privacy and compliance considerations

- Tenant isolation is a release blocker. Enforce authorization server-side and preferably with PostgreSQL row-level security (RLS) where supported by the architecture.
- Never expose service-role keys, provider secrets, webhook signing secrets or privileged server credentials to the browser.
- Verify payment webhook signatures/authenticity, amount, currency, provider reference and order association. Deduplicate events.
- Use secure session handling, CSRF protections where applicable, input validation, output escaping and protection against injection/XSS.
- Validate file type and size; do not trust filename extensions. Store user uploads in controlled paths with safe public/private access.
- Use least privilege for platform and restaurant roles. Require re-authentication or extra checks for sensitive ownership/payment configuration changes if appropriate.
- Collect only the customer information required to fulfill orders. Publish privacy, cancellation and refund terms, and define retention/deletion procedures.
- Protect personal information in logs, analytics and exported reports. Restrict access to customer contact details and payment records.
- Provide audit records for restaurant publication/suspension, role changes, menu/price changes where important, order transitions, payment/refund exceptions and privileged data access.
- Confirm applicable Kenyan data protection, consumer, tax, e-commerce and payment-provider obligations with qualified advisers before live launch; this document is not legal advice.

13. Notifications and user-facing copy

13.1 Required event notifications

Event	Student message / intent	Restaurant message / intent
Order created, payment pending	"We're confirming your payment. Don't place the order again yet."	No fulfilment alert until the configured payment/order rule says it is actionable.
Payment confirmed / order sent	"Payment confirmed. We've sent your order to [Restaurant]."	"New paid order #[number]. Review and accept or reject it."
Restaurant accepts	"[Restaurant] accepted your order. Estimated ready time: [time]."	Confirmation shown in order timeline.
ETA changed	"Your order's estimated ready time changed to [time]. [Reason if supplied]."	Staff sees updated ETA and record of prior value.
Order ready	"Your order is ready for collection. Show code [code] at [Restaurant]."	"Order #[number] is marked ready; verify pickup before marking collected."
Rejected/cancelled	Explain the reason if available and current refund/payment state.	Show rejection/cancellation result and required financial follow-up.
Refund status	"Refund initiated" or "Refund completed" only when the recorded state supports the wording.	Show refund/payment exception status where relevant.

13.2 Copy and messaging rules

- Use "estimated ready time" rather than "guaranteed ready time."
- Use "Ready for collection" instead of vague "Done" when the customer must still pick up the meal.
- Distinguish "Payment pending," "Payment confirmed," "Refund initiated" and "Refund completed."
- Errors should state what happened and the next action: e.g. "This item just became unavailable. Remove it or return to the menu."
- Do not reveal internal exception codes, raw provider payloads or staff-only notes to customers.
- Keep notification templates configurable but versioned. A notification failure must be logged and retried when safe; it must not duplicate an order.

14. Analytics, reporting and audit trail

14.1 Product analytics

Collect a minimal, privacy-conscious event stream to test whether the product solves the actual problem. Useful events include restaurant_viewed, menu_item_viewed, cart_started, checkout_started, order_submitted, payment_confirmed, restaurant_accepted, eta_updated, order_ready, order_collected, order_cancelled, payment_failed and refund_completed. Events should include timestamp, relevant opaque IDs, source/screen and non-sensitive context, not payment secrets or unnecessary personal data.

14.2 Pilot success metrics

Metric	Definition / reason
Order conversion	Completed/valid orders relative to checkout starts or eligible restaurant views; define the denominator consistently.

Metric	Definition / reason
Repeat use	Share of customers who place another order within a selected time period.
Restaurant activation	Share of onboarded restaurants that publish a menu and successfully fulfill at least one order.
Order acceptance time	Time from the order becoming actionable to restaurant acceptance/rejection.
ETA accuracy	Difference between last communicated ETA and ready_at; report median and percentage ready by communicated ETA.
Cancellation/rejection rate	Share of submitted orders cancelled/rejected, segmented by restaurant and reason.
Payment exception rate	Share of payment attempts that require manual reconciliation or enter an unexpected state.
Collection completion	Share of accepted/paid orders that reach collected/completed status.
Restaurant retention	Share of participating restaurants still actively taking orders over time.

14.3 Audit events

Audit events must have actor, timestamp, scope, target, action and reason where applicable. Keep the audit log append-only for normal application roles. Audit event data should be concise and sanitized; do not store entire secrets or unnecessary provider/customer payloads.

15. Edge cases and failure handling

Scenario	Required behaviour
Restaurant closes while cart is open	Revalidate at checkout; block new order with clear explanation. Do not delete the cart silently.
Menu item becomes unavailable	Revalidate on cart update and checkout; identify item and require removal/reselection.
Price changes before checkout	Use server price; display revised total and require user reconfirmation before payment.
Payment provider request times out	Keep payment status pending/unknown until safely queried or webhooked; never assume failure or success from timeout alone.
Duplicate payment callback	Process idempotently; one payment transition/order activation only.
Payment confirmed but order notification fails	Persist the order; retry notification and show an operational alert. Do not charge again.
Restaurant does not respond	Escalate/expire according to configured timeout, notify customer and begin cancellation/refund handling where money was collected.
Restaurant rejects paid order	Mark rejected; initiate approved reversal/refund path and keep separate refund status visible.

Scenario	Required behaviour
Staff loses connectivity	Indicate stale/offline status; allow recovery on reconnect; avoid optimistic final status claims before server confirmation.
Two staff update same order	Use version/status checks; reject illegal stale update and reload the latest state.
Customer double taps pay/place order	Use request idempotency and disable duplicate submissions while processing.
Customer abandons pending payment	Expire according to timeout; keep audit trail and make retry/recovery safe.
Order is ready but not collected	Keep it ready until staff confirms collection/cancellation; later add configurable holding/expiry policy if necessary.
Pickup code guessed/entered incorrectly	Rate limit, give generic error and do not expose another order's details.
Restaurant is suspended	Block new orders/public visibility according to policy; preserve existing order fulfilment, refunds and records.
User removed from restaurant	Invalidate access promptly; future requests must fail even if old UI remains open.
Notification provider fails	Keep app state correct; retry safe events and show in-app status as fallback.
Restaurant menu deleted after past sale	Keep historical order snapshots and report lines intact.

16. Non-functional requirements

Area	Minimum requirement
Responsive UX	Core flows work at common mobile widths and desktop/tablet widths. No horizontal overflow on key screens.
Performance	Aim for fast initial content and responsive interactions on ordinary mobile connections. Paginate/search lists instead of fetching all orders or restaurants.
Availability/recovery	Use managed backups and documented restoration steps. Define recovery expectations before production; payment/order correctness takes priority over cosmetic availability.
Data integrity	Database constraints, transactions where appropriate, idempotent financial event handling and a validated order state machine.
Accessibility	Semantic labels, keyboard support, visible focus, adequate contrast, useful validation messages, status not conveyed by colour alone.
Observability	Structured logs, error capture, uptime checks and alerts for webhook/order delivery failures.
Maintainability	TypeScript, linting, code formatting, migrations, environment validation, modular domain logic and tests for critical paths.
Localization	MVP uses English UI, KES currency formatting and Africa/Nairobi local business time. Store timestamps consistently (UTC recommended) and convert for display.

Area	Minimum requirement
Scalability	Use indexed tenant-scoped queries and pagination. Avoid premature microservices; maintain boundaries that permit later extraction.
Browser support	Test current major Chromium-based browsers and Safari/Firefox where available, plus mobile viewport behaviour.
Security updates	Keep dependencies patched, separate staging/production, use strong access control and rotate compromised secrets promptly.

17. Acceptance criteria and test plan

The implementation is not complete merely because each screen renders. The following end-to-end conditions must pass in a staging environment before the first real pilot.

17.1 Platform and tenant acceptance

- An authorized admin can create two or more restaurants through the same wizard without code changes or redeployment.
- Each published restaurant receives the correct logo, colours, location, hours, items and prices from its own stored configuration.
- Changing a restaurant setting does not change another restaurant's storefront.
- A draft or suspended restaurant is not orderable by the public; any previous orders remain accessible according to policy.
- An owner/staff user from Restaurant A cannot access Restaurant B's private data by guessing or substituting IDs, URLs, API arguments or realtime channel identifiers.
- Public endpoints never reveal customer phone/email, private order notes or internal restaurant/admin configuration.

17.2 Customer and order acceptance

- A customer can browse, choose a restaurant, add an item with modifiers, adjust quantity, checkout, and see a unique order number.
- The server recomputes item prices and total; tampering with client totals or prices cannot change the payable amount.
- The same cart cannot contain items from multiple restaurants without an explicit reset decision.
- Double-submitting checkout/payment does not create duplicate actionable orders or duplicate charges.
- Customer can only view their own order and sees correct state after refresh/reconnect.
- Order details preserve historical product names, modifiers and prices after a menu edit.
- Pickup code is unique/unpredictable enough for its purpose, validated with rate limits and not leaked to unrelated users.

17.3 Restaurant acceptance

- Incoming paid/actionable orders appear only in the correct restaurant queue and create an alert.
- Staff can accept with an ETA, reject with a reason, advance preparing/ready/collected and see who changed the order.
- Illegal state transitions are rejected server-side.
- Changing ETA creates an event and informs the customer through the configured notification path.
- Pausing a restaurant blocks new orders while existing orders remain manageable.
- Menu price/availability edits affect new checkouts but not past order records.

17.4 Payments and reliability acceptance

- Provider callbacks are authenticated/verified and idempotent. Duplicate callbacks do not duplicate fulfilment or payment records.
- A mismatched amount/currency/order reference is not marked as a successful payment and creates a reviewable exception.
- Timeouts and pending provider responses do not cause a blind second charge.
- Rejected/cancelled prepaid orders enter the correct refund/reversal process and show a distinct refund status.
- Critical failure cases are covered by automated tests where practical; manual test cases are documented for sandbox provider flows.
- Logs and monitoring provide enough information to diagnose an issue without exposing secrets or unnecessary personal information.

17.5 Test layers

- Unit tests: total calculation, modifier validation, role checks, ETA suggestions, order/payment transition rules.
- Integration tests: database constraints, RLS/tenant isolation, order creation transaction, callback idempotency, notification dispatch and audit records.
- End-to-end tests: student purchase through collection; admin creates and publishes a restaurant; owner updates menu and processes an order.
- Security tests: cross-tenant access attempts, privilege escalation, parameter tampering, invalid callbacks, brute-force pickup-code attempts and upload validation.
- Operational tests: network loss/reconnect, stale status recovery, provider timeout, webhook retries, backup restoration and production environment verification.

18. Phased delivery plan

Phases are delivery gates, not calendar promises. Final estimates should follow technical review and the owner's decision on payment flow and authentication.

Phase	Scope	Exit condition
0. Product decisions & UX	Confirm auth, order modes, payment/settlement, cancellation/refund policy, fees, business hours and first pilot area. Wireframe the main screens.	Open decisions documented; core data model and flows approved.
1. Foundation	Project setup, design system, auth, role/membership model, database migrations, restaurant tenant boundaries, storage and deployment environments.	Secure login and automated tenant-boundary tests pass.
2. Admin restaurant builder	Create/edit/draft/preview/publish restaurants, branding/configuration, menu builder, owner invitation.	Admin can create multiple distinct restaurants without code changes.
3. Student marketplace	Discovery, profiles, menus, cart, checkout shell, order history and responsive layouts.	Customer can browse and submit a test order in staging.
4. Restaurant operations	Live queue, accept/reject, ETA, prepare/ready/collected, menu availability, pickup code.	Complete test order can move through valid states and updates reach the student.
5. Payments & notifications	Approved payment provider flow, verified callbacks, reconciliation/refund handling, notification templates and retries.	Sandbox payment lifecycle and key exceptions pass tests.
6. Pilot hardening	Security review, accessibility/performance review, operational dashboards, backup/restore check, staff onboarding and pilot analytics.	All release-blocking acceptance criteria pass; support/runbook is ready.

Phase	Scope	Exit condition
7. Post-pilot iteration	Improve ETA logic, scheduled orders, promotions, loyalty, native apps or delivery only from validated demand.	Prioritize features using pilot data and restaurant/student interviews.

19. Decisions to confirm before production

A developer can start the architecture and prototype using the suggested defaults below, but the following business decisions must be explicitly approved before accepting real orders or money.

Decision	Recommended MVP default	Why it matters
Product name/domain	DineFlow as a temporary internal name only.	Brand/trademark/domain availability must be checked before public launch.
Target launch area	One campus and nearby food businesses; Kabarak University is a candidate.	Keeps onboarding and support manageable.
Student authentication	Choose email/password, magic link, phone or guest+verified contact after user research.	Affects checkout conversion, support and account ownership.
Order modes	Pickup first; enable dine-in only if instructions and fulfilment rules are clear.	“Dine-in” can imply a table reservation, which is not included in this MVP.
Payment/settlement	Use an approved provider arrangement; consider pay-at-pickup during a closed pilot if direct settlement is not ready.	Funds flow, merchant onboarding, fees, refunds and compliance cannot be assumed.
Who pays platform fees	Start with a low-friction pilot and track value; decide subscription, commission or hybrid later.	Pricing must be acceptable to small restaurants and financially viable.
Tax/fee display	Show KES totals and disclose every fee. Configure tax only after an accountant/adviser confirms treatment.	Avoid silently inventing tax or fee behaviour.
Cancellation/refunds	Customer self-cancel before restaurant acceptance; later cases handled by restaurant/admin.	Defines operational responsibility and payment automation.
Order acceptance timeout	Set a pilot value after observing kitchen response capacity; notify staff and customer when overdue.	Prevents paid orders from staying unresolved indefinitely.
Customer notifications	In-app/order screen first; select SMS/email/push providers and cost/consent rules.	External notifications can have cost and delivery limitations.
Location/distance	Manual service area or stored restaurant coordinates; ask for device location only with permission.	Do not promise distance sorting without reliable coordinates.
Privacy/data retention	Define policy and deletion/anonymization process with qualified local advice.	Orders and financial records may need a different retention rule from profile data.
Languages	English for MVP; make copy central/configurable for future localization.	Avoid duplicating UI text throughout components.
Admin power boundaries	Define which support staff can view orders, change restaurant status or act on refunds.	Reduces abuse and accidental high-impact actions.

Appendix A. Suggested screen and route inventory

Area	Screen / route concept	Purpose / primary actions
Public	/	Marketplace home, search and filters.
Public	/restaurants/[slug]	Restaurant details and categorized menu.
Public/customer	/cart	Review one-restaurant cart, edit quantities/options.
Customer	/checkout	Contact, pickup/dine-in, payment and final confirmation.
Customer	/orders/[orderNumber]	Order state timeline, ETA, pickup code, receipt/summary.
Customer	/account/orders	Order history and details.
Customer	/account/profile	Profile, contact and notification preferences.
Restaurant	/restaurant	Operational overview/order queue for current assignment.
Restaurant	/restaurant/orders/[id]	Order details, accept/reject, ETA, status transitions and pickup verification.
Restaurant	/restaurant/menu	Categories/items management and availability.
Restaurant	/restaurant/settings	Hours, order mode, pickup instructions, branding fields allowed to owners.
Restaurant	/restaurant/team	Staff invitations and memberships, if enabled.
Restaurant	/restaurant/reports	Scoped order/sales/ETA/payment summaries.
Admin	/admin	Platform overview and exceptions.
Admin	/admin/restaurants	Search, filter, status changes and onboarding.
Admin	/admin/restaurants/new	Restaurant setup wizard.
Admin	/admin/restaurants/[id]	Configuration, preview, publishing, memberships and audit history.
Admin	/admin/orders	Permissioned platform-wide order support view.
Admin	/admin/payments	Payment reconciliation and refund exceptions.
Admin	/admin/users	Platform access and role management.
Admin	/admin/audit	Searchable privileged-action history.
System	/auth/* and provider webhook endpoints	Authentication callbacks and server-to-server payment events.

Appendix B. Definition of done

A feature is done when all applicable conditions below are met:

- Its expected behaviour and permissions are documented and reviewed.
- Loading, empty, success, validation, network-error and permission-denied states are implemented where applicable.
- All sensitive changes are validated server-side; tenant scoping is enforced in the backend/database.
- Critical business rules have automated unit/integration tests, and the main user journey has an end-to-end or documented manual test.
- UI works at mobile and desktop breakpoints and includes accessible labels/focus/error states.
- Logs and monitoring exist for critical external integrations without leaking secrets.
- Migrations, environment configuration and deployment notes are committed.
- Acceptance criteria for the feature pass in staging, and unresolved limitations are recorded.

Final direction to the implementation team

Build a secure, mobile-first, multi-tenant restaurant marketplace and operational system. Make the restaurant creation wizard reusable and data-driven, make the kitchen order lifecycle reliable, treat money movement as a separate auditable state machine, and prioritize a controlled campus pilot over broad feature count.